

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ  
КОММУНИКАЦИЙ  
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение  
высшего образования  
Сибирский государственный университет телекоммуникаций и информатики

Кафедра телекоммуникационных систем и вычислительных средств  
(ТС и ВС)

РАСЧЕТНО-ГРАФИЧЕСКАЯ РАБОТА

по дисциплине

*Визуальное программирование и человеко-машинное взаимодействие*

по теме:

ПРОГРАММНО-АППАРАТНЫЙ КОМПЛЕКС (ПАК)

Студент:

*Группа ИКС-433*

*С.А. Демин*

Преподаватель:

*Старший преподаватель*

*Р.В. Ахнашев*

Новосибирск 2026 г.

## СОДЕРЖАНИЕ

|                                                                                               |    |
|-----------------------------------------------------------------------------------------------|----|
| ВВЕДЕНИЕ .....                                                                                | 4  |
| 1 ТЕОРИЯ.....                                                                                 | 6  |
| 1.1 Географические координаты (LLA) .....                                                     | 6  |
| 1.2 Архитектура GSM и LTE.....                                                                | 6  |
| 1.2.1 GSM (2G) .....                                                                          | 6  |
| 1.2.2 LTE (4G) .....                                                                          | 6  |
| 1.3 Критерии качества радиосигнала .....                                                      | 7  |
| 2 ANDROID ПРИЛОЖЕНИЕ И ЕГО ФУНКЦИОНАЛ .....                                                   | 8  |
| 2.1 Лабораторная работа №6. Местоположение смартфона .....                                    | 8  |
| 2.1.1 Цель .....                                                                              | 8  |
| 2.1.2 Теория .....                                                                            | 8  |
| 2.1.3 Реализация .....                                                                        | 8  |
| 2.2 Лабораторная работа №8. Работа с сокетами (ZMQ). Передача<br>данных от Android к PC ..... | 10 |
| 2.2.1 Цель .....                                                                              | 10 |
| 2.2.2 Теория .....                                                                            | 10 |
| 2.2.3 Реализация .....                                                                        | 10 |
| 2.3 Лабораторная работа №10. Android Cellinfo .....                                           | 12 |
| 2.3.1 Цель .....                                                                              | 12 |
| 2.3.2 Теория .....                                                                            | 12 |
| 2.3.3 Реализация .....                                                                        | 12 |
| 2.4 Лабораторная работа №11. Android background service .....                                 | 14 |
| 2.4.1 Цель .....                                                                              | 14 |
| 2.4.2 Теория .....                                                                            | 14 |
| 2.4.3 Реализация .....                                                                        | 14 |
| 3 BACKEND ПРИЛОЖЕНИЕ И ЕГО ОСНОВНЫЕ ФУНКЦИИ .....                                             | 17 |
| 3.1 Лабораторная работа №9. Backend-сервер на C++.....                                        | 17 |
| 3.1.1 Теория: ZMQ и передача данных между Android и<br>сервером .....                         | 17 |
| 3.1.2 Реализация .....                                                                        | 17 |

|       |                                                              |    |
|-------|--------------------------------------------------------------|----|
| 3.2   | Лабораторная работа №13. Работа с базой данных PostgreSQL .. | 19 |
| 3.2.1 | Теория: база данных и её схема .....                         | 19 |
| 3.2.2 | Реализация .....                                             | 21 |
| 3.3   | Лабораторная работа №14. Отрисовка OpenStreetMap карты ....  | 22 |
| 3.3.1 | Теория: проекция Меркатора и тайлы .....                     | 22 |
| 3.3.2 | Реализация .....                                             | 23 |
| 3.4   | Лабораторная работа №15. Генерация тепловой карты.....       | 25 |
| 3.4.1 | Теория: метод обратных взвешенных расстояний (IDW).          | 25 |
| 3.4.2 | Реализация .....                                             | 26 |
| 4     | ЗАКЛЮЧЕНИЕ .....                                             | 29 |
|       | СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....                        | 31 |

## ВВЕДЕНИЕ

Целью расчетно-графической работы является разработка клиент-серверного приложения, включающего Android-клиент для сбора геолокационных данных и параметров радиосигнала, а также backend-сервер на языке C++ для приёма, хранения в базе данных PostgreSQL, визуализации на OpenStreetMap картах и генерации тепловых карт на основе метода обратных взвешенных расстояний.

Для достижения поставленной цели в работе решаются следующие задачи:

- Разработка Android-приложения, обеспечивающего получение данных о местоположении.
- Реализация в Android-приложении сбора информации о сотах сотовой связи стандартов LTE, GSM и NR посредством класса Telephony.
- Организация фонового сбора данных и передачи их на сервер через сокеты ZeroMQ.
- Разработка backend-сервера на языке C++ с многопоточной архитектурой: отдельный поток для работы ZMQ-сокетов и отдельный поток для графического интерфейса пользователя.
- Интеграция с СУБД PostgreSQL для сохранения всех входящих данных в структурированную таблицу.
- Реализация модуля загрузки и отображения тайлов OpenStreetMap с динамическим изменением в зависимости от положения камеры, включая кэширование загруженных изображений.
- Построение тепловых карт для критериев RSRP, RSRQ, RSSI с возможностью выбора отображаемого параметра через GUI.

В ходе работы были использованы следующие инструменты и технологии: язык Kotlin для написания Android-приложения и C++ для серверной части; библиотека ZeroMQ для обмена данными между телефоном и компьютером; libpq для подключения к базе данных PostgreSQL; libcurl для скачивания карт из интернета; ImGui и ImPlot для создания окон и графиков; GitHub для хранения кода и отслеживания изменений.

В результате получился работающий программно-аппаратный комплекс, который в реальном времени собирает данные о сигнале сотовой сети, помогает оценить качество покрытия и позволяет проводить замеры в городе с последующим анализом собранной информации.

# 1 ТЕОРИЯ

## 1.1 Географические координаты (LLA)

LLA (Latitude, Longitude, Altitude) - система координат, используемая для определения положения точки на поверхности Земли. Широта - это угол между плоскостью экватора и отвесной линией в данной точке. Измеряется в градусах, изменяется от  $-90^\circ$  (южная широта) до  $+90^\circ$  (северная широта). Долгота - угол между плоскостью нулевого меридиана и плоскостью меридиана, проходящего через точку. Измеряется в градусах от  $-180^\circ$  до  $+180^\circ$ . Высота - расстояние от точки до среднего уровня моря, измеряется в метрах. [1]

## 1.2 Архитектура GSM и LTE

### 1.2.1 GSM (2G)

GSM — стандарт второго поколения для голосовых вызовов, СМС и низкоскоростного интернета. Технология сохраняет актуальность благодаря множеству устройств с поддержкой только 2G [2].

Архитектура: мобильная станция (UE), базовая станция (BTS), контроллер BSC, центр коммутации (MSC-S/MGW). Множественный доступ — TDMA/FDMA. Пакетную передачу данных обеспечивает узел Gn/Gp SGSN.

### 1.2.2 LTE (4G)

LTE — наиболее востребованная технология, обеспечивающая высокие скорости интернета, поддержку VoLTE и NB-IoT [2].

Ключевые компоненты: UE, базовая станция (eNodeB), узел управления мобильностью (MME), обслуживающий шлюз (SGW), пакетный шлюз (PGW с интерфейсом Gn), сервер абонентских данных (HSS), узел политик тарификации (PCRF) [2].

### 1.3 Критерии качества радиосигнала

Received Signal Strength Indicator (RSSI) — индикатор уровня принимаемого сигнала. Измеряется в дБм. Показывает общую мощность сигнала, включая полезный сигнал, шумы и помехи. Для LTE: [3]

- Отлично:  $> -65$  дБм
- Хорошо:  $-65 \dots -75$  дБм
- Удовлетворительно:  $-75 \dots -85$  дБм
- Плохо:  $-85 \dots -95$  дБм
- Нет сигнала:  $\leq -95$  дБм

Reference Signal Received Power (RSRP) — мощность опорных (пилотных) сигналов от базовой станции. Измеряется в дБм. Характеризует уровень полезного сигнала без учёта шумов. Для LTE: [3]

- Отлично:  $\geq -80$  дБм
- Хорошо:  $-80 \dots -90$  дБм
- Удовлетворительно:  $-90 \dots -100$  дБм
- Плохо:  $\leq -100$  дБм

## 2 ANDROID ПРИЛОЖЕНИЕ И ЕГО ФУНКЦИОНАЛ

### 2.1 Лабораторная работа №6. Местоположение смартфона

#### 2.1.1 Цель

Получить доступ к данным о местоположении Android-телефона и вывести на экран значения.

#### 2.1.2 Теория

Для определения местоположения в Android используются GPS, Wi-Fi и сотовые вышки. Класс `FusedLocationProviderClient` объединяет данные от всех источников, обеспечивая оптимальный баланс точности и энергопотребления. Для доступа к координатам требуется запросить разрешения `ACCESS_FINE_LOCATION` или `ACCESS_COARSE_LOCATION` [4].

#### 2.1.3 Реализация

В проекте создано `Activity location.kt`. При запуске запрашиваются разрешения, инициализируется `FusedLocationProviderClient` [5]:

```
1 class location: AppCompatActivity() {
2     lateinit var myFusedLocationProviderClient: FusedLocationProviderClient
3
4     override fun onCreate(savedInstanceState: Bundle?) {
5         super.onCreate(savedInstanceState)
6         myFusedLocationProviderClient = LocationServices.
            getFusedLocationProviderClient(this) }
```

Listing 2.1 --- Инициализация  
`FusedLocationProviderClient`

Метод `getCurrentLocation()` получает координаты с высокой точностью:

```
1 private fun getCurrentLocation() {
2     if (checkPermissions()) {
3         if (isLocationEnabled()) {
4             myFusedLocationProviderClient.getCurrentLocation(Priority.
                PRIORITY_HIGH_ACCURACY, null).addOnCompleteListener(this) { task
                ->
```



```

5         val location:Location?=task.result
6         if (location!=null){
7             lat_site=location.latitude
8             lon_site=location.longitude
9             lat.setText(location.latitude.toString())
10            lon.setText(location.longitude.toString())
11            alt.setText(round(location.altitude).toString())
12            tojson(location.latitude,location.longitude,location.
                altitude,location.time.toString())}}}}

```

Listing 2.2 --- Получение текущего местоположения

Данные выводятся в TextView, а также сохраняются в файл location.json в папке Downloads. Кнопка card\_button открывает карту в браузере:

```

1 card_button.setOnClickListener({
2     val uri=Uri.parse("https://geotree.ru/coordinates?lat=$lat_site&lon=
        $lon_site")
3     val siteIntent=Intent(Intent.ACTION_VIEW,uri)
4     startActivity(siteIntent)})

```

Listing 2.3 --- Открытие карты

Обновление координат происходит каждые 10 секунд:

```

1 private fun updatelocation(){
2     handler.postDelayed({
3         getCurrentLocation()
4         updatelocation(), 10000})

```

Listing 2.4 --- Периодическое обновление

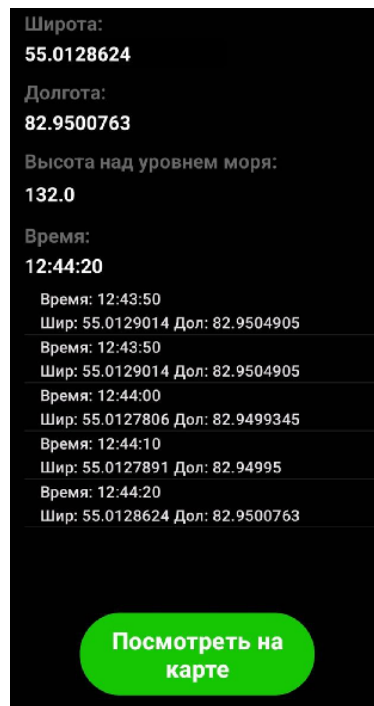


Рисунок 1 — Экран Location

## 2.2 Лабораторная работа №8. Работа с сокетами (ZMQ). Передача данных от Android к PC

### 2.2.1 Цель

Реализовать обмен сообщениями между Android-приложением и сервером на ПК с использованием ZeroMQ [6].

### 2.2.2 Теория

ZeroMQ - высокопроизводительная асинхронная библиотека для обмена сообщениями, поддерживающая различные паттерны. В отличие от обычных сокетов, ZMQ автоматически управляет буферизацией, переподключением и маршрутизацией. Для взаимодействия Android-клиента с ПК-сервером используется сокет типа REQ (запрос) и REP (ответ).

### 2.2.3 Реализация

Создано Activity sockets.kt. При запуске запускается поток сервера, который связывается с портом 6666:

```

1 fun startServer() {
2     val context=ZMQ.context(1)
3     val socket=ZContext().createSocket(SocketType.REP)
4     socket.bind("tcp://localhost:6666")
5     var counter:Int=0
6
7     while (true){
8         counter++
9         val requestBytes=socket.recv(0)
10        val request=String(requestBytes,ZMQ.CHARSET)
11        handler.postDelayed({serverView.text=request},0)
12        val response="Получено сообщение №$counter"
13        socket.send(response.toByteArray(ZMQ.CHARSET),0) } }

```

Listing 2.5 --- Серверная часть на ZMQ

Клиент отправляет текст из EditText при нажатии кнопки. Переключатель позволяет менять адрес сервера на внешний IP:

```

1 fun startClient() {
2     val context=ZMQ.context(1)
3     val socket=ZContext().createSocket(SocketType.REQ)
4     if (flag){socket.connect("tcp://192.168.1.211:2222")
5     }else{socket.connect("tcp://localhost:6666") }
6     val request=clientText.text.toString()
7     socket.send(request.toByteArray(ZMQ.CHARSET),0)
8     val reply=String(socket.recv(0),ZMQ.CHARSET)
9     clientView.text=reply}

```

Listing 2.6 --- Клиентская часть

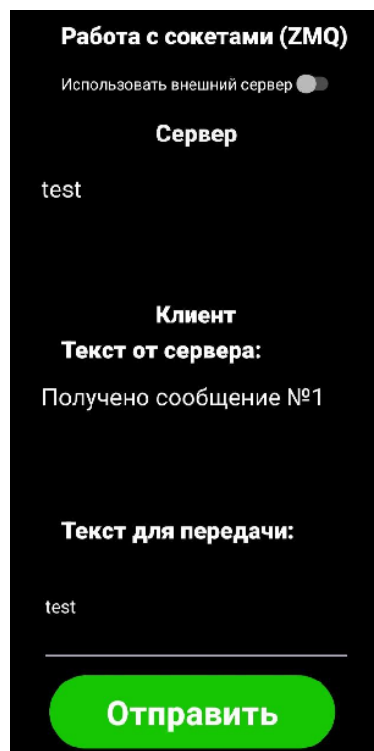


Рисунок 2 — Экран Sockets

## 2.3 Лабораторная работа №10. Android Cellinfo

### 2.3.1 Цель

Получать и передавать на сервер детальную информацию о сотах сотовой связи вместе с геолокацией и сетевым трафиком.

### 2.3.2 Теория

Класс `TelephonyManager` [7] предоставляет метод `getAllCellInfo()`, возвращающий список сотовых вышек текущего типа сети (LTE, GSM, NR). Для каждой соты можно получить уникальные идентификаторы (CID, TAC, PCI, EARFCN) и параметры сигнала (RSRP, RSRQ, RSSNR). Эти данные вместе с GPS-координатами позволяют анализировать покрытие сети и качество связи.

### 2.3.3 Реализация

Создано Activity `telephony.kt`. Пользователь выбирает тип сети (LTE, GSM, NR) через кнопки:

```

1 btn_lte.setOnClickListener({
2     flag=1
3     CellIdentity.setText("CellIdentityLTE")
4     CellSignalStrength.setText("CellSignalStrengthLTE")
5     btn_lte.setBackgroundColor(Color.parseColor("#2d2d2f"))
6     btn_gsm.setBackgroundColor(Color.parseColor("#16c603"))
7     btn_nr.setBackgroundColor(Color.parseColor("#16c603")) })

```

Listing 2.7 --- Выбор типа сети

### Получение информации о сотах LTE:

```

1 if (flag==1){
2     val telephonyManager=getSystemService(Context.TELEPHONY_SERVICE) as
        TelephonyManager
3     val cellInfoList=telephonyManager.allCellInfo
4     cellInfoList?.forEach{cellInfo->
5         if (cellInfo is CellInfoLte){
6             cellIdentityList.add("Band:␣${cellInfo.cellIdentity.bandwidth}")
7             cellIdentityList.add("CellIdentity:␣${cellInfo.cellIdentity.ci}")
8             cellIdentityList.add("EARFCN:␣${cellInfo.cellIdentity.earfcn}")
9             cellIdentityList.add("PCI:␣${cellInfo.cellIdentity.pci}")
10            cellIdentityList.add("TAC:␣${cellInfo.cellIdentity.tac}")
11            cellSignalStrengthList.add("RSRP:␣${cellInfo.cellSignalStrength.
                rsrp}")
12            cellSignalStrengthList.add("RSRQ:␣${cellInfo.cellSignalStrength.
                rsrq}")
13            cellSignalStrengthList.add("RSSI:␣${cellInfo.cellSignalStrength.
                rssi}") }}}

```

Listing 2.8 --- Получение данных о сотах LTE

### Данные отображаются в двух списках (идентификаторы и мощность):

```

1 val a1=ArrayAdapter(this,android.R.layout.simple_list_item_1,cellIdentityList)
2 list_cell_info1.adapter=a1
3 val a2=ArrayAdapter(this,android.R.layout.simple_list_item_1,
        cellSignalStrengthList)
4 list_cell_info2.adapter=a2

```

Listing 2.9 --- Отображение данных в ListView

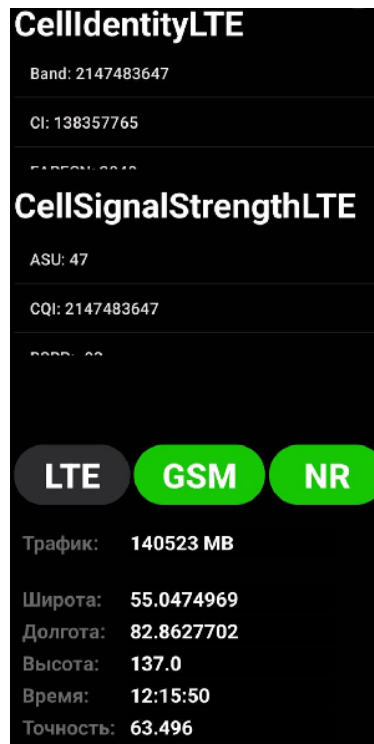


Рисунок 3 — Экран Cellinfo

## 2.4 Лабораторная работа №11. Android background service

### 2.4.1 Цель

Реализовать фоновый сервис, который в фоне собирает данные о местоположении, сотах и трафике, отправляет их на сервер и сохраняет в файл.

### 2.4.2 Теория

Сервисы Android позволяют приложению выполнять код в фоне, даже когда экран выключен или пользователь работает с другим приложением. Это нужно для периодического сбора местоположения, данных о сотах и отправки их на сервер без участия пользователя.

### 2.4.3 Реализация

Класс `BackgroundService.kt` наследуется от `Service`. В методе `onStartCommand` запускается корутина, в которой циклически вызываются `getCurrentLocation()` и `getCellInfo()`:

```
1 class BackgroundService:Service() {
```

```

2 private val serviceJob=Job()
3 private val serviceScope=CoroutineScope(Dispatchers.IO+serviceJob)
4
5 override fun onStartCommand(intent:Intent?, flags:Int, startId:Int):Int{
6     flag=intent?.getIntExtra("flag", 1)?:1
7     serviceScope.launch{
8         while (isActive){
9             getCurrentLocation()
10            getCellInfo()
11            delay(3000) }}
12     return START_STICKY}}

```

Listing 2.10 --- Фоновый сервис – основной цикл

### Отправка данных в Activity через LocalBroadcastManager:

```

1 private fun sendLocationToActivity(location:Location){
2     val intent=Intent("LOCATION_UPDATE")
3     intent.putExtra("latitude",location.latitude.toString())
4     intent.putExtra("longitude",location.longitude.toString())
5     intent.putExtra("altitude",round(location.altitude).toString())
6     intent.putExtra("accuracy",location.accuracy.toString())
7     val totalTraffic=TrafficStats.getTotalRxBytes()+TrafficStats.
8         getTotalTxBytes()
9     intent.putExtra("traffic","${totalTraffic/1024/1024}MB")
10    LocalBroadcastManager.getInstance(this).sendBroadcast(intent)}

```

Listing 2.11 --- Отправка данных в Activity

### Формирование JSON и отправка на сервер через ZMQ:

```

1 private fun sendAllData(location:Location){
2     val jsonData=JSONObject()
3     val locData=JSONObject()
4     locData.put("latitude",location.latitude)
5     locData.put("longitude",location.longitude)
6     locData.put("altitude",round(location.altitude))
7     jsonData.put("location",locData)
8     val cellsArray=JSONArray()
9     jsonData.put("cells",cellsArray)
10    Thread{
11        val socket=ZContext().createSocket(SocketType.REQ)
12        socket.connect("tcp://172.22.237.35:443")
13        socket.send(jsonData.toString().toByteArray(ZMQ.CHARSET),0)
14        socket.close().start()
15    }.saveToFile(jsonData)}

```

Listing 2.12 --- Отправка данных на сервер

### Сохранение данных в файл:

```

1 private fun saveToFile(jsonData:JSONObject){

```

```

2      val downloadsDir=Environment.getExternalStoragePublicDirectory(Environment
      .DIRECTORY_DOWNLOADS)
3      val file=File(downloadsDir,"mydata.json")
4      val jsonObject=if (file.exists()) JSONObject(file.readText()) else
      JSONObject()
5      val dataArray=if (jsonObject.has("data")) jsonObject.getJSONArray("data")
      else JSONArray()
6      dataArray.put(jsonData)
7      jsonObject.put("data",dataArray)
8      file.writeText(jsonObject.toString()) }

```

Listing 2.13 --- Сохранение в JSON файл

### Запуск и остановка сервиса из Activity:

```

1      bStart.setOnClickListener{
2          val startBgIntent=Intent(this,BackgroundService::class.java)
3          startBgIntent.putExtra("flag",flag)
4          startService(startBgIntent) }
5
6      bStop.setOnClickListener{
7          val stopBgIntent=Intent(this,BackgroundService::class.java)
8          stopService(stopBgIntent) }

```

Listing 2.14 --- Управление сервисом

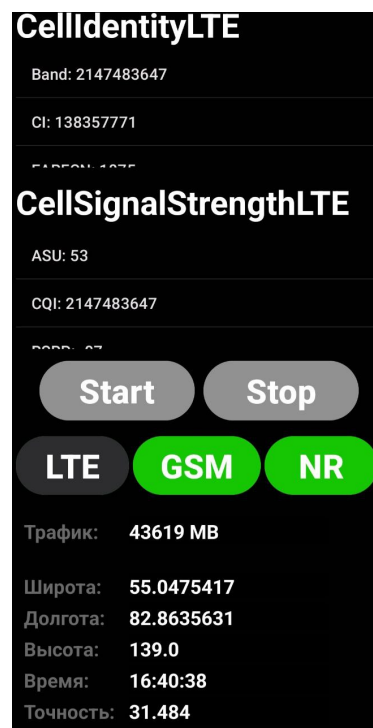


Рисунок 4 — Экран location+cellinfo+traffic



## 3 BACKEND ПРИЛОЖЕНИЕ И ЕГО ОСНОВНЫЕ ФУНКЦИИ

### 3.1 Лабораторная работа №9. Backend-сервер на C++

#### 3.1.1 Теория: ZMQ и передача данных между Android и сервером

ZeroMQ - это библиотека для асинхронного обмена сообщениями между процессами или по сети. Она работает поверх обычных сокетов, но упрощает программирование: не нужно управлять соединениями, повторными отправлениями и буферизацией. В проекте используется шаблон REQ/REP. Android выступает в роли клиента, отправляя JSON с координатами на сервер, а сервер отвечает коротким подтверждением. Данные передаются по TCP. Сервер привязан к порту 5555 на всех интерфейсах, клиент подключается по IP-адресу компьютера в той же локальной сети [6].

#### 3.1.2 Реализация

Сервер написан на C++ с использованием библиотек: libzmq для сокетов, nlohmann/json для парсинга JSON, ImGui и ImPlot для графического интерфейса, SDL2 как бэкенд окна.

#### Структуры данных

Для передачи данных из серверного потока в GUI-поток определена структура LocationData с координатами, высотой и временем, а также мьютекс для потокобезопасного доступа.

```
1 struct LocationData{
2     float latitude=0.0f;
3     float longitude=0.0f;
4     float altitude=0.0f;
5     float accuracy=0.0f;
6     std::string time="No_data";
7     long long time_milliseconds=0;
8     TrafficData traffic;
9     std::vector<CellTowerData> cellTowers;
10    std::mutex mutex;
11 };
```

Listing 3.1 --- Структура LocationData

## Серверный поток

Функция `run_zmq_server()` создаёт ZMQ-контекст и сокет типа REP, привязывает его к порту 443. В бесконечном цикле сокет ожидает сообщение от клиента. При получении JSON-строки она парсится, извлекаются поля с координатами, временем и данными о сотах. Для потокобезопасного доступа используется мьютекс. Сервер также сохраняет данные в JSON-файл и базу данных.

```
1 void run_zmq_server(LocationData* loc){
2     zmq::context_t context(1);
3     zmq::socket_t socket(context,ZMQ_REP);
4     socket.bind("tcp://*:443");
5
6     while(true){
7         zmq::message_t request;
8         socket.recv(request);
9         std::string msg=static_cast<char*>(request.data());
10
11         try{
12             json j=json::parse(msg);
13             { std::lock_guard<std::mutex> lock(loc->mutex);
14                 if (j.contains("location")&&j["location"].is_object()){
15                     auto& loc_data=j["location"];
16                     loc->latitude=getFloatValue(loc_data,"latitude",0.0f);
17                     loc->longitude=getFloatValue(loc_data,"longitude",0.0f);
18                     loc->altitude=getFloatValue(loc_data,"altitude",0.0f);
19                     loc->accuracy=getFloatValue(loc_data,"accuracy",0.0f);
20                     loc->time=loc_data.value("time","No_data");}}
21             g_measurementCounter++;
22             saveToJsonFile(*loc,g_measurementCounter);
23             updateSignalHistory(*loc);}
24         catch(const json::parse_error& e){std::cerr<<"Ошибка JSON: " <<e.what()
                <<std::endl;}}
```

Listing 3.2 --- Обработка входящих данных на сервере

## GUI-поток

Функция `run_gui()` инициализирует окно SDL2, контекст OpenGL, ImGui и ImPlot. В каждом кадре отображается окно с заголовком LOCATION & TELEPHONY, где выводятся значения из структуры LocationData.

## Запуск потоков

В функции `main()` создаются два потока: `gui_thread` и `server_thread`. Им передаётся указатель на общую структуру `LocationData`.

```
1 int main() {
2     if (g_database.connect()) {std::cout<<"\033[32mПодключением к БД
    успешно\033[0m"<<std::endl;}
3     std::thread gui_thread(run_gui,&g_locationData);
4     std::thread server_thread(run_zmq_server,&g_locationData);
5     gui_thread.join();
6     server_thread.join();}
```

Listing 3.3 --- Запуск потоков в `main()`

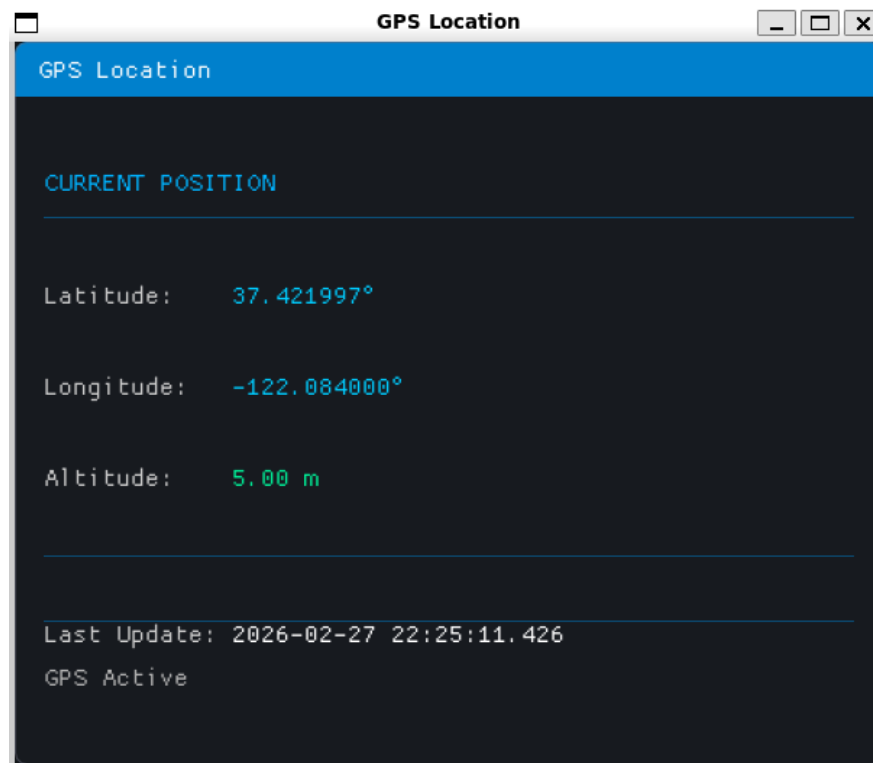


Рисунок 5 — Окно сервера на C++ 1 версии

## 3.2 Лабораторная работа №13. Работа с базой данных PostgreSQL

### 3.2.1 Теория: база данных и её схема

PostgreSQL - реляционная система управления базами данных. В проекте используется для хранения всех измерений, поступающих от Android-приложения. Схема базы данных состоит из двух таблиц: `based` (основные изме-

рения) и cells (данные о сотах). Связь один-ко-многим: одно измерение может содержать информацию о нескольких вышках сотовой связи.

```
1 CREATE TABLE based (  
2     id SERIAL PRIMARY KEY,  
3     counter INTEGER NOT NULL,  
4     "current_time" BIGINT NOT NULL,  
5     time TEXT NOT NULL,  
6     latitude REAL NOT NULL,  
7     longitude REAL NOT NULL,  
8     accuracy REAL,  
9     altitude REAL,  
10    traffic_total BIGINT,  
11    traffic_total_rx BIGINT,  
12    traffic_total_tx BIGINT  
13 );  
14  
15 CREATE TABLE cells (  
16     id SERIAL PRIMARY KEY,  
17     measurement_id INTEGER NOT NULL,  
18     type TEXT,  
19     mcc INTEGER,  
20     mnc INTEGER,  
21     cell_identity BIGINT,  
22     tac INTEGER,  
23     earfcn INTEGER,  
24     band INTEGER,  
25     pci INTEGER,  
26     rsrp INTEGER,  
27     rsrq INTEGER,  
28     rssi INTEGER,  
29     rssnr INTEGER,  
30     asu_level INTEGER,  
31     cqi INTEGER,  
32     timing_advance INTEGER,  
33     FOREIGN KEY (measurement_id) REFERENCES based(id)  
34 );  
35  
36 CREATE INDEX idx_based_time ON based(time);  
37 CREATE INDEX idx_based_location ON based(latitude, longitude);  
38 CREATE INDEX idx_cells_measurement ON cells(measurement_id);  
39 CREATE INDEX idx_cells_mcc_mnc ON cells(mcc, mnc);  
40 CREATE INDEX idx_cells_type ON cells(type);
```

Listing 3.4 --- Схема базы данных PostgreSQL

### 3.2.2 Реализация

Сервер на C++ подключается к PostgreSQL через библиотеку libpq. В классе Database реализованы методы connect(), insertMeasurement(), insertCell().

```
1 class Database{
2 public:
3     Database():conn(nullptr){}
4     ~Database(){if (conn) PQfinish(conn);}
5     bool connect(){
6         const char* info="host=" DB_HOST "port=" DB_PORT
7             "dbname=" DB_NAME "user=" DB_USER
8             "password=" DB_PASSWORD;
9         conn=PQconnectdb(info);
10        if (PQstatus(conn)!=CONNECTION_OK){
11            std::cerr<<"ОШИБКА подключения:"<<PQerrorMessage(conn)<<"\n";
12            return false;}return true;}
13
14    int insertMeasurement(const LocationData& data,int counter);
15    bool insertCell(int measurement_id,const CellTowerData& cell);};
```

Listing 3.5 --- Класс Database для работы с PostgreSQL

При получении JSON от Android сервер парсит данные, вызывает insertMeasurement для записи в таблицу based, а затем для каждой соты - insertCell.

```
1 int Database::insertMeasurement(const LocationData& data,int counter){
2     std::ostringstream query;
3     query << "INSERT INTO based (counter,current_time,time,"
4         << "latitude,longitude,accuracy,altitude,"
5         << "traffic_total,traffic_total_rx,traffic_total_tx)"
6         << "VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9,$10)"
7         << "RETURNING id";
8     return measurement_id;}
9
10 bool Database::insertCell(int measurement_id,const CellTowerData& cell){
11     std::ostringstream query;
12     query << "INSERT INTO cells (measurement_id,type,mcc,mnc,"
13         << "cell_identity,tac,earfcn,band,pci,rsrp,rsrq,"
14         << "rssi,rssnr,asu_level,cqi,timing_advance)"
15         << "VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9,$10,"
16         << "$11,$12,$13,$14,$15,$16)";}
```

Listing 3.6 --- Вставка данных в БД

В GUI-потоке данные о сигналах накапливаются в структуре SignalHistory с мьютексом. Графики используют библиотеку ImPlot.

```

1 struct SignalHistory{
2     std::vector<double> timestamps;
3     std::map<int,std::vector<double>> rsrp_values;
4     std::map<int,std::vector<double>> rsrq_values;
5     std::map<int,std::vector<double>> rssi_values;
6     std::map<int,std::vector<double>> rssnr_values;
7     std::map<int,std::vector<double>> ss_rsrp_values;
8     std::map<int,std::vector<double>> ss_rsrq_values;
9     std::map<int,std::vector<double>> ss_sinr_values;
10    std::map<int,std::vector<double>> dbm_values;
11    std::mutex mutex;};

```

Listing 3.7 --- Структура для хранения истории сигналов

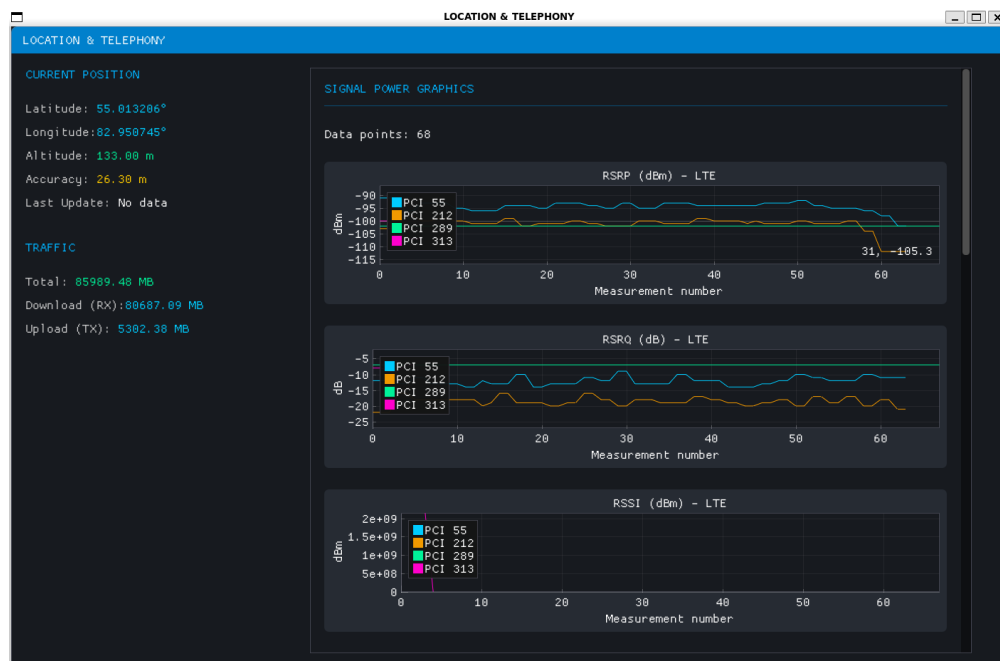


Рисунок 6 — Окно сервера на C++ 2 версии

### 3.3 Лабораторная работа №14. Отрисовка OpenStreetMap карты

#### 3.3.1 Теория: проекция Меркатора и тайлы

Для отображения карты на экране используется проекция Меркатора. Все карты OpenStreetMap разбиты на квадратные тайлы размером 256 × 256 пикселей. Каждый тайл однозначно идентифицируется уровнем масштаба zoom и координатами x, y.

### 3.3.2 Реализация

#### Класс TileManager

Создан класс `TileManager`, который управляет загрузкой, кэшированием и отображением тайлов OSM. Основные возможности: асинхронная загрузка тайлов из сети, кэширование на диске, декодирование PNG в RGBA с использованием `libpng`, создание текстур OpenGL.

```
1 bool TileManager::downloadTile(int zoom, int x, int y, std::vector<uint8_t>&
   rgbaData, int& width, int& height){
2     std::string url="https://tile.openstreetmap.org/"+std::to_string(zoom)+"/"
       +std::to_string(x)+"/"+std::to_string(y)+".png";
3     CURL* curl=curl_easy_init();
4     curl_easy_setopt(curl, CURLOPT_URL, url.c_str());
5     curl_easy_setopt(curl, CURLOPT_WRITEFUNCTION, WriteCallback);
6     CURLcode res=curl_easy_perform(curl);
7     //обработка ответа и декодирование png
8     return decodePngToRgba(response, rgbaData, width, height); }
```

Listing 3.8 --- Метод загрузки тайла

#### Пересчёт координат и подгрузка тайлов

В GUI-потоке при отображении карты определяются текущие границы видимой области. На основе этих границ вычисляется оптимальный уровень масштаба `zoom`.

```
1 int TileManager::getZoomForLimits(double minLon, double maxLon){
2     double diff=maxLon-minLon;
3     if (diff>90.0) return 4;
4     else if (diff>45.0) return 5;
5     else if (diff>22.5) return 6;
6     else if (diff>11.25) return 7;
7     else return 15; }
8
9 double TileManager::mercatorXToTileX(double mercatorX, int zoom){
10     return (0.5+mercatorX/360.0)*(1<<zoom); }
11
12 double TileManager::mercatorYToTileY(double mercatorY, int zoom){
13     double lat_rad=mercatorY*M_PI/180.0;
14     double y_normalized=0.5-log(tan(M_PI/4.0+lat_rad/2.0))/(2.0*M_PI);
15     return y_normalized*(1<<zoom); }
```

Listing 3.9 --- Пересчёт географических координат в тайлы

## Отображение карты в ImPlot

Карта рисуется с помощью функции `ImPlot::PlotImage()`, которая принимает текстурный идентификатор и координаты углов тайла.

```
1 void TileManager::renderTiles(double minX, double maxX, double minY, double maxY)
2 {
3     int zoom=getZoomForLimits(minX,maxX);
4     for (int x=minTileX;x<=maxTileX;x++){
5         for (int y=minTileY;y<=maxTileY;y++){
6             GLuint gpuId=getTextureId(zoom,x,y);
7             if (gpuId!=0){
8                 ImPlotPoint minPoint{tileXToMercatorX(x,zoom),tileYToMercatorY
9                     (y+1,zoom)};
10                 ImPlotPoint maxPoint{tileXToMercatorX(x+1,zoom),
11                     tileYToMercatorY(y,zoom)};
12                 ImPlot::PlotImage(("##tile_"+tileId).c_str(),(ImTextureID)(
13                     intptr_t)gpuId,minPoint, maxPoint);}}}}
```

Listing 3.10 --- Рендеринг тайлов карты

## Асинхронная загрузка

Для плавной работы интерфейса загрузка тайлов выполняется в отдельном потоке через очередь задач.

```
1 void TileManager::fetchWorker() {
2     while (m_running){
3         TileJob job;
4         bool hasJob=false;
5         { std::lock_guard<std::mutex> lock(m_jobMutex);
6             if (!m_jobQueue.empty()){
7                 job = m_jobQueue.front();
8                 m_jobQueue.pop();
9                 hasJob=true;}}
10        if (hasJob){
11            std::vector<uint8_t> rgbaData;
12            int width, height;
13            if (downloadTile(job.zoom,job.x,job.y,rgbaData,width,height)){
14                }else{std::this_thread::sleep_for(std::chrono::milliseconds(10));}}
```

Listing 3.11 --- Асинхронная загрузка тайлов



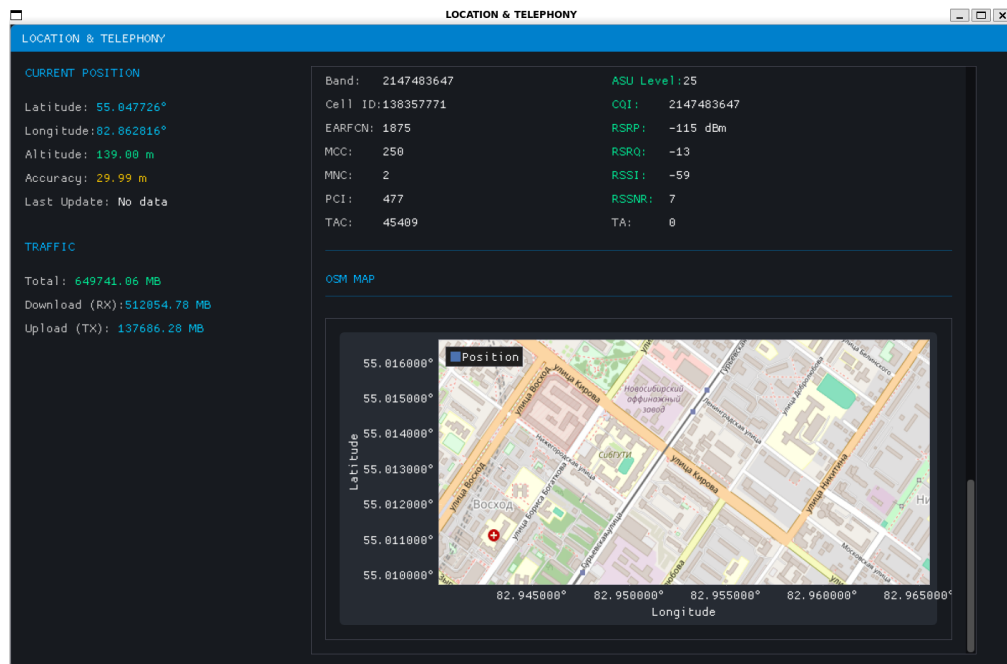


Рисунок 7 — Окно сервера на C++ 3 версии

### 3.4 Лабораторная работа №15. Генерация тепловой карты

#### 3.4.1 Теория: метод обратных взвешенных расстояний (IDW)

Тепловая карта строится на основе интерполяции экспериментальных измерений. Используется метод обратных взвешенных расстояний (IDW, Inverse Distance Weighting):

$$\text{weight}(x) = \frac{\sum_{i=1}^N \frac{\text{weight}_i}{d(x, x_i)^p}}{\sum_{i=1}^N \frac{1}{d(x, x_i)^p}}$$

где  $d(x, x_i)^p$  — расстояние между точками, а при  $d(x, x_i) = 0$  значение  $\text{weight}(x)$  принимается равным  $\text{weight}_i$ .

Критерии оценки RSRP:

- Excellent: > -80 dBm (красный цвет)
- Good: от -80 до -90 dBm
- Fair/Marginal: от -90 до -100 dBm
- Poor/Weak: от -100 до -110 dBm (тёмно-синий)
- No Signal: < -110 dBm (не закрашивается)

### 3.4.2 Реализация

#### Класс HeatmapData

Создан класс HeatmapData, который загружает из базы данных PostgreSQL все измерения LTE. Данные группируются по метрике и по EARFCN. Для каждого набора вычисляется динамический диапазон значений.

```
1 std::vector<HeatmapDbRow> Database::getHeatmapRows() {
2     std::vector<HeatmapDbRow> rows;
3     const char* query=
4         "SELECT b.latitude, b.longitude, b.accuracy, b.altitude, c
5         "c.earfcn, c.rsrp, c.rsrq, c.rssi"
6         "FROM basedb"
7         "JOIN cells ON c.measurement_id=b.id"
8         "WHERE c.type='LTE'"
9         "ORDER BY b.id";
10
11     PGresult* res=PQexec(conn, query);
12     for (int i=0; i<PQntuples(res); ++i) {
13         HeatmapDbRow row;
14         row.latitude=std::stod(PQgetvalue(res, i, 0));
15         row.longitude=std::stod(PQgetvalue(res, i, 1));
16         row.earfcn=std::stoi(PQgetvalue(res, i, 4));
17         row.rsrp=std::stoi(PQgetvalue(res, i, 5));
18         rows.push_back(row); } return rows; }
```

Listing 3.12 --- Загрузка данных для тепловой карты из БД

#### Рендеринг тайлов тепловой карты

Метод renderTile() для заданного тайла создаёт изображение 256×256 пикселей. Для каждого пикселя вычисляются географические координаты, затем по методу IDW интерполируется значение.

```
1 bool HeatmapData::renderTile(const HeatmapSelection& selection, int zoom, int x,
2     int y, std::vector<uint8_t>& rgbaData, int& width, int& height) const {
3     width=256; height=256;
4     rgbaData.assign(width*height*4, 0);
5
6     for (int pixelY=0; pixelY<height; ++pixelY) {
7         const double latitude=tilePixelToLatitude(zoom, y, pixelY, height);
8         for (int pixelX=0; pixelX<width; ++pixelX) {
9             const double longitude=tilePixelToLongitude(zoom, x, pixelX, width);
10             double weightedSum=0.0;
```

```

10         double totalWeight=0.0;
11         for (const Sample* sample:candidates){
12             const double distance=haversineMeters(latitude,longitude,
                sample->latitude,sample->longitude);
13             const double weight=1.0/std::pow(distance,selection.power);
14             weightedSum+=sample->value*weight;
15             totalWeight+=weight;}
16
17         if (totalWeight>0.0){
18             double interpolatedValue=weightedSum/totalWeight;}}}
19     return hasVisiblePixels;}

```

Listing 3.13 --- Интерполяция IDW для тепловой карты

## Цветовая схема

Для визуализации значений используется градиент от синего (худшие значения) через зелёный и жёлтый к красному (лучшие значения).

```

1 void HeatmapData::colorize(double ratio,float opacity,uint8_t& r,uint8_t& g,
    uint8_t& b,uint8_t& a){
2     static const ColorStop stops[]={
3         {0.0,10.0,20.0,120.0},
4         {0.35,0.0,120.0,255.0},
5         {0.7,255.0,200.0,0.0},
6         {1.0,255.0,0.0,0.0}};
7     r=static_cast<uint8_t>(left->r+(right->r-left->r)*localRatio);
8     g=static_cast<uint8_t>(left->g+(right->g-left->g)*localRatio);
9     b=static_cast<uint8_t>(left->b+(right->b-left->b)*localRatio);
10    a=static_cast<uint8_t>(255.0f*opacity);}

```

Listing 3.14 --- Функция цветовой схемы

## Интеграция в GUI

В окне ImGui добавлена панель управления тепловой картой с выбором метрики, EARFCN и параметров интерполяции.

```

1 ImGui::Checkbox("Show heatmap",&heatmapEnabled);
2 const char* metricItems[]={ "RSRP", "RSRQ", "RSSI", "Altitude" };
3 ImGui::Combo("Criterion",&selectedMetric,metricItems,IM_ARRAYSIZE(metricItems)
    );
4
5 std::vector<int> availableEarfcns=g_tileManager.getAvailableHeatmapEarfcns(
    metric);
6 if (ImGui::BeginCombo("EARFCN",std::to_string(selectedEarfcn).c_str())){
7     for (int earfcn:availableEarfcns){

```

```

8         if (ImGui::Selectable(std::to_string(earfcn).c_str(),selectedEarfcn==
           earfcn)){
9             selectedEarfcn=earfcn;
10            heatmapGenerated=false;}} ImGui::EndCombo();}
11
12 if (ImGui::Button("Generate_heatmap")){
13     heatmapGenerated=true;
14     g_tileManager.clearHeatmapCache();}
15
16 g_tileManager.setHeatmapSelection({
17     heatmapEnabled && heatmapGenerated,
18     metric, selectedEarfcn,radiusMeters,
19     displayRadiusMeters,2.0,0.55f});

```

Listing 3.15 --- Панель управления тепловой картой в GUI

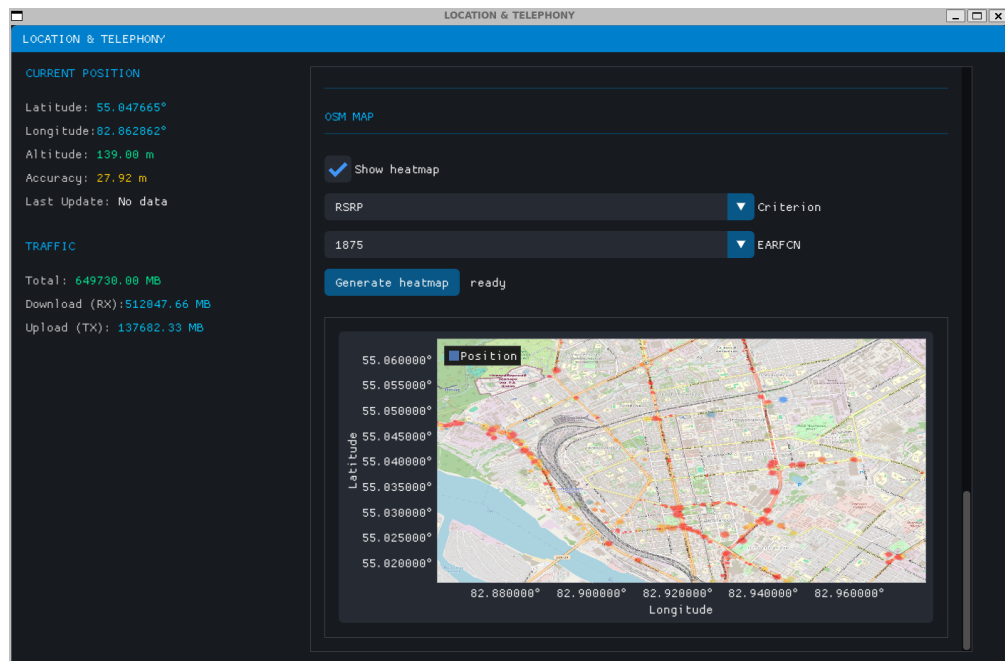


Рисунок 8 — Окно сервера на C++ 4 версии

## 4 ЗАКЛЮЧЕНИЕ

За учебный год в ходе выполнения лабораторных работ я освоил следующие навыки и технологии.

### Android-разработка

- Работа с геолокацией: получение координат (широта, долгота, высота), точности, текущего времени с помощью FusedLocationProviderClient.
- Использование разрешений ACCESS\_FINE\_LOCATION, ACCESS\_COARSE\_LOCATION, READ\_PHONE\_STATE.
- Сбор информации о сотовых сетях через TelephonyManager: данные LTE, GSM, NR.
- Создание фонового сервиса для сбора данных в фоне, с запуском и остановкой из Activity.
- Передача данных между сервисом и Activity через LocalBroadcastManager.
- Сохранение данных в JSON-файлы во внутреннее хранилище.
- Подключение к ZMQ-серверу и отправка JSON-строк.
- Разработка MediaPlayer: воспроизведение mp3 из внешней памяти, управление громкостью, SeekBar, список треков.
- Создание нескольких Activities и переходы между ними, обработка нажатий кнопок.

### Backend-разработка

- Написание сервера на C++ с использованием библиотеки ZeroMQ.
- Парсинг входящих JSON-сообщений с помощью библиотеки nlohmann/json.
- Многопоточность: отдельный поток для ZMQ-сервера и отдельный поток для графического интерфейса, синхронизация через мьютексы и общую структуру LocationData.
- Сохранение полученных данных в JSON-файл с накоплением истории.
- Работа с базой данных PostgreSQL: подключение через libpq, создание таблиц based и cells, вставка измерений и данных о сотах.

- Визуализация графиков сигналов для разных PCI с помощью библиотеки ImPlot, цветовая дифференциация.
- Отображение карт OpenStreetMap: асинхронная загрузка тайлов через libcurl, декодирование PNG в RGBA, создание текстур OpenGL.
- Реализация динамической подгрузки тайлов при изменении масштаба и панорамировании, кэширование на диске.
- Генерация тепловых карт методом обратных взвешенных расстояний по критериям RSRP, RSRQ, RSSI, высоте. Наложение полупрозрачной тепловой карты поверх OSM.
- Настройка параметров интерполяции.

## Инструменты

- Работа с системой контроля версий Git: создание веток, коммиты, слияния, push в удалённый репозиторий.
- Ведение README.md с описанием проекта, инструкциями по сборке и запуску.
- Использование библиотек: ZeroMQ, nlohmann/json, libpq, libcurl, libpng, SDL2, ImGui, ImPlot.
- Взаимодействие Android-приложения с сервером через TCP/IP в локальной сети.

Таким образом, я приобрёл практические навыки разработки полноценного программно-аппаратного комплекса для сбора, передачи, хранения и визуализации данных мобильного телефона, что позволяет в дальнейшем применять эти знания для реальных задач мониторинга качества сотовой связи.

## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. *mksegment.ru*. Расшифровка координат долгота широта: понимание системы географических координат // <https://mksegment.ru/c/rasshifrovka-koordinat-dolgota-shirota-osnovnye-ponyatiya-i-primenenie>. — 2025.
2. *Бухтеев М.* 2G и 4G с нами надолго: обзор основных архитектур сетей операторов связи // <https://habr.com/ru/companies/yadro/articles/881320/>. — 2025.
3. *gsm-repiteri.ru*. Что такое RSSI, SINR, RSRP, RSRQ? Параметры качества сигнала // <https://gsm-repiteri.ru/chto-takoe-rssi-sinr-rsrp-rsrq-parametry-kachestva-signala>. —.
4. *Developers A.* Запросить разрешения на определение местоположения // <https://developer.android.com/develop/sensors-and-location/location/permissions?hl=> 2026.
5. *Developers A.* FusedLocationProviderClient // <https://developers.google.com/android/> 2024.
6. *ZeroMQ.* C++ // <https://zeromq.org/languages/cplusplus/>. — 2024.
7. *Developers A.* TelephonyManager // <https://developer.android.com/reference/android/> 2026.